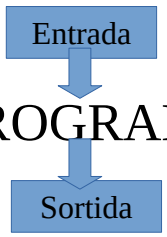
ASPECTES BÀSICS DE PYTHON



Índex de continguts

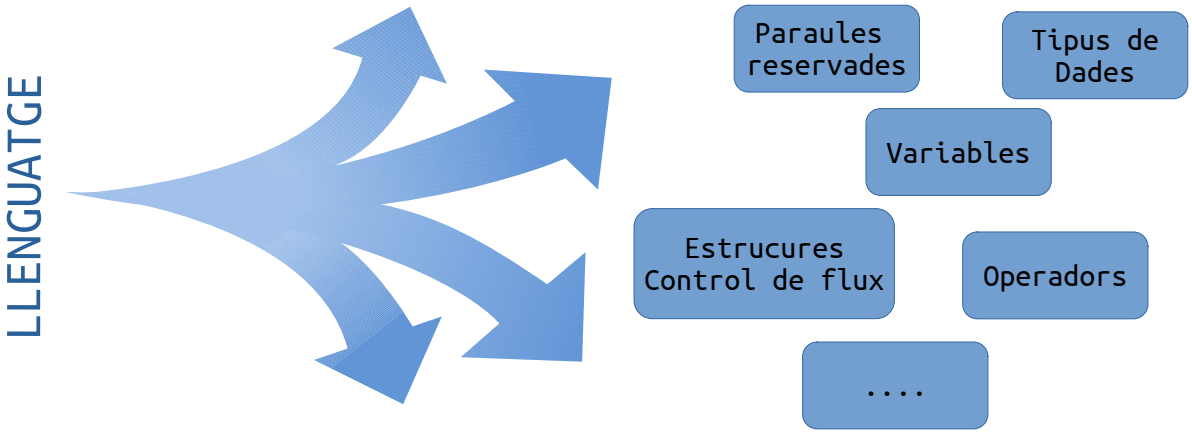
1. DOCUMENTACIÓ OFICIAL....3
2. LLENGUATGE INTERPRETAT....3
3. PARAULES RESERVADES (Python3)....4
4. VARIABLES (magatzem de valors en Variables)....4
5. TIPUS DE DADES BÀSICS....4
   5.1. Tipus NUMÈRICS....5
   5.2. Tipus BOOLEANS....5
   5.3. CONVERSIÓ ENTRE TIPUS....6
6. OPERADORS....6
7. ASSIGNACIÓ....9
8. FUNCIÓ print()....9
9. FUNCIÓ input()....10
10. BLOCS de codi - IDENTACIÓ....11
   10.1. *Condicional*....11
   10.2. Bucle....11
11. ERRORS COMUNS....13
12. SCRATCH vs PYTHON....14

Versió 10/10/20023



PROGRAMA == CONJUNT D'INSTRUCCIONS

Un **programa** és un conjunt de passos lògics [algorismes/receptes] escrits en un llenguatge de programació que ens permet realitzar una tasca específica.



1. DOCUMENTACIÓ OFICIAL

https://docs.python.org/3/

2. LLENGUATGE INTERPRETAT

PYTHON és un llenguatge interpretat

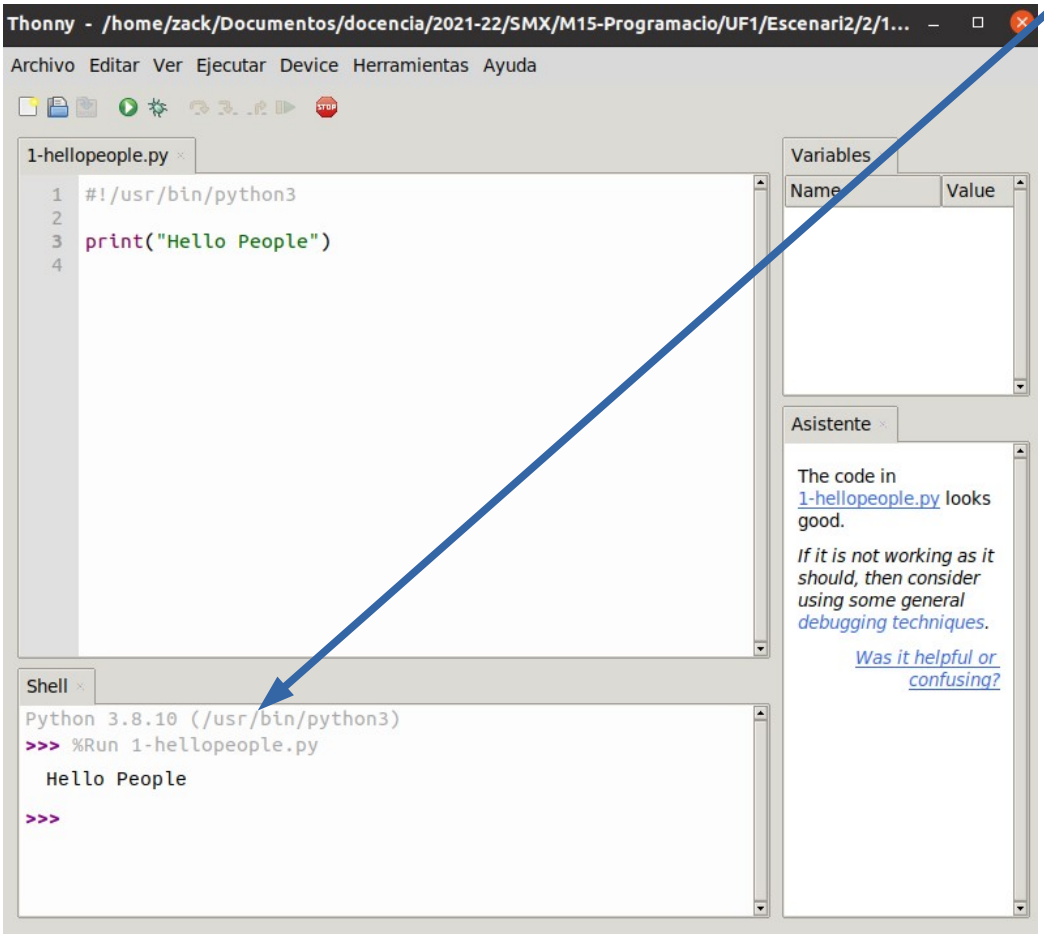
Un programa Python es pot executar en diferents modes: «Terminal» o *Consola* i *Script (fitxer)*

```
zack@aula:~$ python
Python 2.7.18 (default, Mar  8 2021, 13:02:45)
[GCC 9.3.0] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Hello People!")
Hello People!
>>>
```

```
zack@aula:~$ python3 hellopeople.py
Hello People
zack@aula:~$
```

Consola de Python
>>>

UTILITZACIÓ D'UN **IDE**



PROGRAMA+BÀSIC: EXECUTAR INSTRUCCIONS

INSTRUCCIONS BÀSIQUES: print, type, operacions, assignacions variables, ...

3. PARAULES RESERVADES (Python3)

Les paraules reservades no es poden utilitzar com identificadors de variables.

```
and       del       from      not       while
as        elif      global    or        with
assert    else      if        pass      yield
break     except    import    print
class     exec      in        raise
continue  finally   input     return
def       for       is        try
False     None      lambda    True       nonlocal
```

Utilització de # pels comentaris

Evitar noms de variable amb ...:
- accents, espais, o caràcters especials
- variables només amb una lletra (a, b, ...)
- sense significat (hla, aa, a1, a2, ...)

4. VARIABLES (magatzem de valors en Variables)

En Python no és necessari declarar les variables abans d'utilitzar-les. L'assignació inicial crea la variable i enllaça el valor a la variable. Ull perquè diferencia entre majúscules i minúscules!

• Exemples en la ***consola*** de Python:

```
>>> x = 8                 # Create a variable x by assigning a value
>>> x = 'Hello'           # Re-assign a value (of a different type) to x
>>> print('Valor de x:', X) '   # Error de variable no definida! (vigila amb majúscules-minúscules)
```

5. TIPUS DE DADES BÀSICS

str	**int**	**long**	**float**	**bool**
text="Hello People"	**entero = 23**	**enterllarg = 23L**	**real = 0.2703**	**certfals=False**

• Exemples en la ***consola*** de Python:

```
>>> help(str)                 # veure ajuda

>>> cadena = "Hola Mundo"     # cadenas de caracteres
>>> type(cadena)              # comprobar tipo de dato
<class 'str'>
>>> entero = 23               # número entero
>>> type(entero)              # comprobar tipo de dato
<class 'int'>
>>> certfals = False          # valor booleano
>>> type(certfals)            # comprobar tipo de dato
```

• Python es tipado dinámico, esto es, asocia tipos con objetos en lugar de variables. Así, una variable no tiene un tipo fijo y se le puede asignar un objeto de cualquier tipo. Una variable solo proporciona una referencia a un objeto.

• No es necesario declarar una variable. Una variable se crea automáticamente cuando un valor es asignado la primera vez, que enlaza el objeto a la variable. Se puede usar la función implícita type(nombre_var) paraobtener el tipo de objeto referenciado por una variable.

• La funció **input** per llegir de teclat, obté les dades de tipus string (**str**).

• Exemples en la ***consola*** de Python:

```
>>> x = 1                  # Assign an int value to create variable x
>>> x                      # Display x
1

>>> print (type(x))        # Get the type of x
<class 'int'>              # Re-assign x to a float


>>> x = 1.0
>>> x
1.0
>>> print(type(x))         # Show the type
<class 'float'>


>>> x = 'hello'     # Re-assign x to a string
>>> x
'hello'
>>> print(type(x))         # Show the type
<class 'str'>


>>> x = '123'              # Re-assign x to a string (of digits)
>>> x
'123'
>>> print(type(x))         # Show the type
<class 'str'>
```

```
EXEMPLE DE NOMS DE VARIABLES:

PreuCompra = 10.3
preu_compra = 10.3


Velocitat = 100
velocitatTren = 230


strDNI = ‘45621187F’
str_DNI = ‘45621187F’
int_DNI = 45621187


iva21 = 21 / 100
```

5.1. Tipus NUMÈRICS

Clase	Tipo	Notas	Ejemplo
`int`	Números	Número entero con precisión fija.	`42`
`long`	Números	Número entero en caso de overflow.	`42L` ó `45696678615198764 3L`
`float`	Números	Coma flotante de doble precisión.	`3.1415927`
`complex`	Números	Parte real y parte imaginaria *j*.	`(4.5 + 3j)`

5.2. Tipus BOOLEANS

Tipus o classe: `bool` **Possibles valors: True o False (majúscules són importants!)**
En el contexto de las operaciones booleanas, y también cuando las expresiones son usadas bajo sentencias de flujo de control, los siguientes valores son interpretados como False:

- False
- None
- Número cero en todos los tipos.
- Cadena de caracteres vaciás.

```
>>> girar = False
>>> type(girar)
>>> esGroc = True
>>> type(esGroc)
```

Valor especial per inicialitzar:

```
>>> x = None
```

5.3. CONVERSIÓ ENTRE TIPUS

Text numèric a número

```
>>> meutextnumeric=input('introdueix un número:')
>>> print(meutextnumeric, type(meutextnumeric))
>>> print(meutextnumeric, type(int(meutextnumeric)))
```

Número a text

```
>>> numero=50
>>> print(numero, type(numero))
>>> print(numero, type(str(numero)))
```

Booleà a numèric

>>> **int(False)**
0
>>> **int(True)**
1

La funció **input** en llegir de teclat, obté les dades de tipus string (**str**).

Moltes vegades ens caldrà convertir a sencer (**int)**

cadena_cinc = ”5”

numero_cinc = int(”5”)

6. OPERADORS

Els següents «tokens» són **operadors** de diferents tipus:

```
+       -       *       **      /       //      %
<<      >>      &       |       ^       ~
<       >       <=      >=      ==      !=
```

OPERADORS ARITMÈTICS

Operador	Descripción	Ejemplos
+	Addition	
-	Subtraction	
*	Multiplication	
/	Float Division (returns a float)	1 / 2 ⇒ 0.5 -1 / 2 ⇒ -0.5
//	Integer Division (returns the floor integer) Cocient de la divisió	1 // 2 ⇒ 0 -1 // 2 ⇒ -1 8.9 // 2.5 ⇒ 3.0 -8.9 // 2.5 ⇒ -4.0 -8.9 // -2.5 ⇒ 3.0
**	Exponentiation	2 ** 5 ⇒ 32 1.2 ** 3.4 ⇒ 1.858729691979481
%	Modulus (Remainder) Reste (residu) de la divisió	9 % 2 ⇒ 1 -9 % 2 ⇒ 1 9 % -2 ⇒ -1 -9 % -2 ⇒ -1

Exemples en la ***consola*** de Python:

>>> 2 + 4 **>>> 2 - 4** **>>> -5 – 4** **>>> 5*4**
>>> 24** **>>> 3/4** **>>> 3 / / 4** **>>> 3 % 4**

OPERADORS RELACIONALS (BOOLEANS)

Són utilitzats per avaluar condicionals; com a resposta obtenim ***booleans***.

Símbol	Operació	Exemple	Resultat
==	Igual que	5==5	True
!=	Diferent que	4!=5	True
>	Major que	98>100	False
<	Menor que	12>=39	False
>=	Major o Igual	6>=6	True
<=	Meno o Igual	3<=8	True

Exemples en la *consola* de Python:

```
>>> 3 < 4        >>> 3 == 4        >>> 3 > 4        >>>  5!=2        >>>  5!=5
```

OPERADORS LÒGICS (BOOLEANS)

Permeten construir expressions lògiques, obtenim com a resultat ***booleans***.

Símbol	Operació	Exemple	Resultat
and	Conjunció	12>2 **and** 5<10	True
or	Disjunció	9!=6 **or** 8<=5	True
not	Negació	**not** True	False

OPERADORS LÒGICS – TAULES DE VERITAT (AND, OR, NOT)

Operador lògic AND

x	**y**	**resultat (x and y)**
True	True	True
True	False	False
False	True	False
False	False	False

x és una condició amb operador relacional amb resultat booleà True o False

y és una condició amb operador relacional amb resultat booleà True o False

Exemple:

```
>>> x = 5 > 3
>>> y = 'a' == 'b'
>>> resultat = x and y
>>> print(resultat)
```

Operador lògic OR

x	**y**	**resultat (x or y)**
True	True	True
True	Talse	True
Talse	True	True
Talse	False	False

Exemple:

```
>>> x = 5 > 3
>>> y = ‘a’ == ‘b’
>>> resultat = x or y
>>> print(resultat)
```

Operador lògic NOT

x	**resultat (not x)**
True	False
False	True

Exemple:

```
>>> x = 5 > 3
>>> resultat = not x
>>> print(resultat)
>>> x = ‘a’ == ‘b’
>>> resultat = not y
>>> print(resultat)
```

7. ASSIGNACIÓ

Instrucció bàsica: l'assignació de valors. Podem trobar aquest tipus d'instrucció bàsica de diferents formes utilitzant elements del llenguatge que ja hem vist.

• Exemples en la ***consola*** de Python:

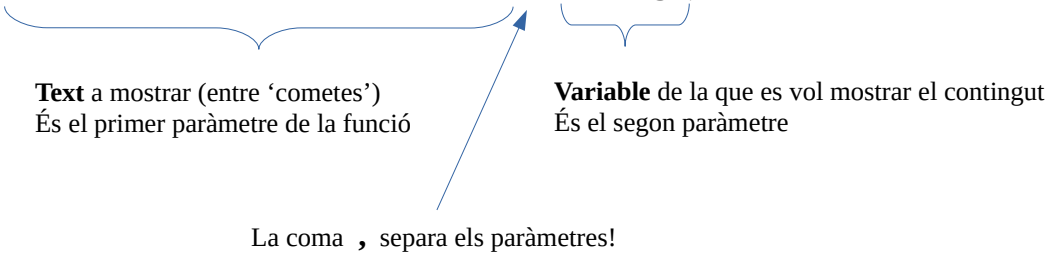
```
>>> numero = 9          # ''valor fix''
>>> sumaNumeros = 3 + 9       #  càlculs
>>> pesMaxim = pes + 9     #  càlculs amb altres variables
>>> numero = int ( '5' )   #  resultat d'una funció que retorna valor
>>> salutacio = input ( 'Indica una salutació:' )   #  resultat d'una funció que retorna valor
# Passem com a paràmetre de la funció «int» (convertir a sencer) el resultat de la funció «input»
>>> numero = int ( input ( 'Indica una salutació:' ) )   #  resultat d'una funció que retorna valor
```

8. FUNCIÓ print()

print És una *funció* pre-definida (built-in function) de Python. Es pot utilitzar de moltes formes per mostrar resultats per pantalla. En general, el text a mostrar es posa entre 'cometes' (dobles o simples). Anem a veure alguns exemples de com utilitzar-la.

```
>>> print('Hola que tal?')
>>> missatge = 'Hola que tal?'
>>> print(missatge)

>>> print('Mostrar el valor d'una variable:', missatge)
```



Una **funció** sempre inclou els parèntesis, i si cal, dintre del parèntesis es passen els paràmetres separats per comes.

```
#######################################################
### E X E M P L E S f-string a partir Python 3.6 ###
#######################################################
>>> missatge = ‘Hola que tal?’
# Mostrar el valor d’una variable: Hola que tal?
>>> print(f‘Mostrar el valor d’una variable:{missatge}’)
```

F-string de **python3**

Variable de la que es vol mostrar el valor (entre claus)

Observa que el missatge a mostrar apareix entre comentes (dobles o simples), i només s’utilitza les *claus* per fer referencia a la variable de la qual es vol mostrar el seu valor. És **obligatori** posar la **f** al davant de la cadena de text a mostrar. Viem un altre exemple:

```
>>> nom = "Juan"
>>> edat = 20
>>> alcada = 1.75
>>> Print(f"Te llamas {nom}, tens {edat} anys i mesures {alcada} metres.")
# Te llamas Juan, tens 20 anys i mesures 1.75 metres.
```

9. FUNCIÓ input()

input És una *funció* pre-definida (built-in function) de Python. Espera un valor teclejat per teclat. Es pot utilitzar de diferents formes, i junt amb la funció *print* permeten la interacció amb l’ordinador. En general, en la instrucció *input* es pot posar un text a mostrar per pantalla (entre ‘cometes’ dobles o simples) com a paràmetre abans d’esperar el valor a introduir per teclat. Anem a veure un exemples de com utilitzar-la.

```
>>> nom = input ( ‘Indica el teu nom:’ )
#  Mostra per pantalla el missatge i espera un valor introduït per teclat, que es guarda en la variable «nom»

>>> print(‘Bon dia ’, nom)
# Mostrar el valor obtingut per teclat i assignat a la variable «nom»
```

10. BLOCS de codi - IDENTACIÓ

Per defecte no és poden posar espais davant de les instruccions! La identació (tabulació) només es poden posar quan hi ha «instruccions de bloc».

Off the top of my head, `def`, `if`, `elif`, `else`, `try`, `except`, `finally`, `with`, `for`, `while`, and `class` all start blocks.

Un block comença just després dels «dos punts» :
cal posar una identació (tabulació, espais)

10.1. *Condicional*

Exemple: **Sentència de control condicional**. Comparar dos valor i imprimir quin és el més gran.

```
if 5 > 10:
    # Només si es compleix condició anterior (5 > 10)
    print('El 5 és més gran que el 10!')
else:
    # ... mostrar en cas contrari (else)
    print('El 5 és menor o igual que el 10!')
```

10.2. `Bucle`

Exemple: **Sentència de control de bucle** (Iterativa) amb **for**. Contar de 0 a 49 (50 valors), i imprimir el número i el seu doble.

```
for x in range(0, 50):
    print(x)   # Això s'executa totes les vegades que dura el bucle: 50 iteracions
    print(2*x) # Això també s'executa 50 vegades

print(x)       # Imprimir últim valor de x del bucle.
```

Exemple: **Sentència de control de bucle** (Iterativa) amb **while**. Contar de 0 a 49 (50 valors), i imprimir el número i el seu doble.

```
X = 0   # Inicialitzar la Variable comptador
while x <= 49:
    print(x)   # Això s'executa totes les vegades que dura el bucle: 50 iteracions
    print(2*x) # Això també s'executa 50 vegades
    x = x + 1  # Això permet que la variable sigui «comptador» d'un en un

print(x)       # Imprimir últim valor de x del bucle.
```

Una explicació, aclariment de com funciona un "bucle" (loop) de la instrucció de control de flux **for**.

Si tenim aquest exemple:

```
for n in (1, 10, 100):
      print("%3d" % n)
```

Això significa que la variable n agafa els valors del rang especificat que en aquest cas són només 3 valors concrets: (1, 10, 100).

Per tant, les instruccions que hi han "dintre" del bucle (identat), es farà per cada valor que agafa la variable "n"

Anem a veure les diferents "iteracions" (repeticions, vegades que es repeteix el codi dintre del "bucle" for)

n=1 (1ª iteració o repetició)

```
print("%3d" % 1)    # Això imprimeix "  1"
```

n=10 (2ª iteració o repetició)

```
print("%3d" % 10)   # Això imprimeix " 10"
```

n=100 (3ª iteració o repetició)

```
print("%3d" % 100)    # Això imprimeix "100"
```

El resultat en pantalla seria (alineant números):

```
  1
 10
100
```

Espero que hagi quedat més clar com funciona el "**bucle** for"

11. ERRORS COMUNS

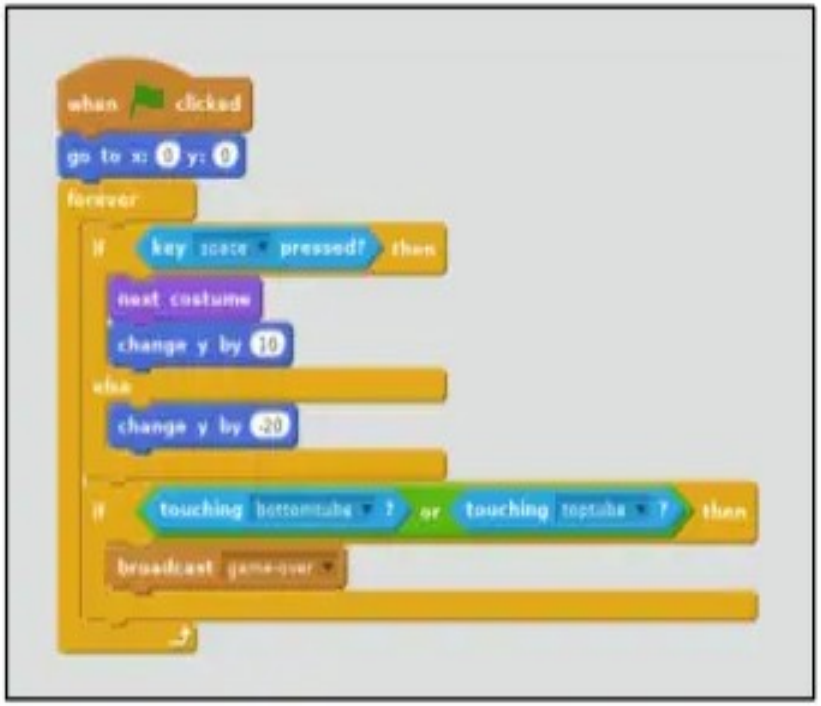
https://www.dummies.com/programming/python/the-8-most-common-python-programming-errors/

https://inventwithpython.com/blog/2012/07/09/16-common-python-runtime-errors-beginners-find/

- **Using the incorrect indentation:** Many Python features rely on indentation. For example, when you use a «for», «if» sentence, or create a new class, everything in that class is indented under the class declaration. The same is true for decision, loop, and other structural statements. If you find that your code is executing a task when it really shouldn't be, start reviewing the indentation you're using.
- **Relying on the assignment operator instead of the equality operator:** When performing a comparison between two objects or value, you just use the equality operator (==), not the assignment operator (=). The assignment operator places an object or value within a variable and doesn't compare anything.
- **Placing function calls in the wrong order when creating complex statements:** Python always executes functions from left to right. So the statement `MyString.strip().center(21, "*")` produces a different result than `MyString.center(21, "*").strip()`. When you encounter a situation in which the output of a series of concatenated functions is different from what you expected, you need to check function order to ensure that each function is in the correct place.
- **Misplacing punctuation:** You can put punctuation in the wrong place and create an entirely different result. Remember that you must include a colon at the end of each structural statement. In addition, the placement of parentheses is critical. For example, `(1 + 2) * (3 + 4)`, `1 + ((2 * 3) + 4)`, and `1 + (2 * (3 + 4))` all produce different results.
- **Using the incorrect logical operator:** Most of the operators don't present developers with problems, but the logical operators do. Remember to use `and` to determine when both operands must be `True` and `or` when either of the operands can be `True`.
- **Creating count-by-one errors on loops:** Remember that a loop doesn't count the last number you specify in a range. So, if you specify the range `[1:11]`, you actually get output for values between 1 and 10.
- **Using the wrong capitalization:** **Python is case sensitive**, so MyVar is different from myvar and MYVAR. Always check capitalization when you find that you can't access a value you expected to access.
- **Making a spelling mistake:** Even seasoned developers suffer from spelling errors at times. Ensuring that you use a common approach to naming variables, classes, and functions does help. However, even a consistent naming scheme won't always prevent you from typing **`MyVer`** when you meant to type **`MyVar`**.

12. SCRATCH vs PYTHON

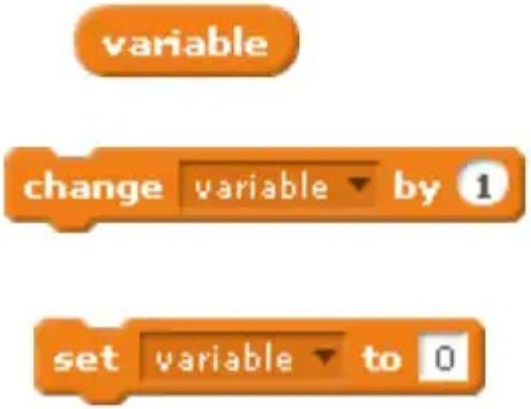
https://www.slideshare.net/EmilydelaPena1/looking-at-how-scratch-and-python-compare




```
1 import turtle
2
3 painter = turtle.Turtle()
4
5 painter.pencolor("blue")
6
7 for i in range(50):
8     painter.forward(50)
9     painter.left(123) # Let's go counterclockwise this time
0
1 painter.pencolor("red")
```

VARIABLES

Scratch:



In Python variables have to be given an initial value:

```
variable = variable + 1
```

```
variable = 0
```

ARITHMETIC

Scratch:



In Python the modulus operator or "mod" is symbolised by a "%" symbol. We use it to get the remainder of a division

```
var1 + var2
var1 - var2
var1 * var2
var1 / var2
var1 % var2
round(var1)
```

OPERATORS

Scratch:



Python:

```
var1 < var2
var1 == var2
var1 > var2
var = (True) and (False)
var = (True) or (False)
var = not (False)
```

CONDITIONALS

Scratch:



Python:

```
if (True):
        # Do something
else:
        # Do something else
```

```
if (True):
        # Do something
```

LOOPS

Scratch:



Python:

```
for i in range(10):
    # Do something
```

```
while (True):
    # Do something forever
```

```
while(not condition) # Waits until
                     # condition is true
```

```
exit()
```

```
while(not condition): # Repeats until condition is true
    # Do something
```

RANDOM NUMBERS

Scratch:



Python:

```
import random
randnum = random.randint(1,10)
```

Random is a whole library of functions.
We have so much more we can do with random numbers in Python than in Scratch!

LISTS

Scratch:



Python:

```
List = []
```

Lists in Python start at index 0

Let's add some objects to our lists




```
List.append(thing)
del List[0]   or   List.pop(0)
List.insert(0, thing)
List[0] = thing
List[0]
```

LISTS

Scratch:



Python:

```
len(List)
```

```
thing in List
```

We can do so much more in Python than in Scratch. We can sort the list, reverse the order of the list etc...

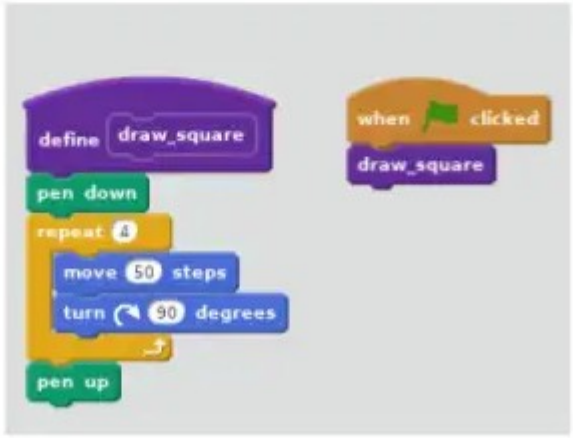
```
List.reverse()
```

```
List.sort()
```

FUNCTIONS

Scratch:





Python:

```
def draw_square():
```

```
import turtle
turtle = turtle.Turtle()

def draw_square():
        for i in range(1,4):
                turtle.forward(10)
                turtle.left(90)

draw_square()
```

CÓMO FUNCIONA

EL INTÉRPRETE DE PYTHON



EDITOR



ARCHIVO DE FUENTE

INTÉRPRETE PYTHON



CORRECTOR DE SINTAXIS Y TRADUCTOR



LIBRERÍAS DE PYTHON



CÓDIGO BYTE



MÁQUINA VIRTUAL PYTHON



PROGRAMA EN EJECUCIÓN